Microservices with Spring Boot 4.0.1

Day 1 Laboratory Manual

Building Your First RESTful Microservice

Technology Stack

- Java 21
- Spring Boot 4.0.1
- Maven
- Spring Data JPA
- MySQL
- Validation API
- Postman

Learning Outcomes

At the end of Day 1, you will be able to:

- Explain the differences between Monolithic and Microservices architectures.
- Create a Spring Boot 4.0.1 project.
- Build a layered Spring Boot application.
- Develop REST APIs using Spring Boot.
- Test APIs using Postman.

Table of Contents

Module 1 – From Monolithic to Microservices

Learning Objectives

After completing this module, you will be able to:

- Explain why software architecture is important.
- Understand Monolithic Architecture.
- Understand Microservices Architecture.
- Compare both architectures.
- Identify appropriate use cases for each.

1.1 Enterprise Applications

An **Enterprise Application** is software developed to support the business operations of an organization. These applications are expected to process large volumes of data, support many concurrent users, and remain available around the clock.

Examples include:

- E-Commerce Platforms
- Banking Systems
- Hospital Management Systems
- University Management Systems
- Airline Reservation Systems
- Inventory Management Systems

Unlike desktop applications, enterprise applications must satisfy several non-functional requirements.

Requirement	Description
Scalability	Handle increasing numbers of users and requests
Availability	Remain accessible with minimal downtime
Performance	Respond quickly under normal and peak loads
Security	Protect sensitive business and customer data
Reliability	Continue operating correctly even under failures
Maintainability	Support enhancements and bug fixes efficiently

As applications grow in complexity, the choice of software architecture becomes critical.

1.2 Evolution of Software Architecture

Software architecture has evolved to address increasing business requirements and technological advancements.

Desktop Applications

Client-Server Architecture

Three-Tier Architecture

Monolithic Architecture

Microservices Architecture

Desktop Applications

Business logic, user interface, and data are contained within a single application installed on one computer.

Client-Server Architecture

The client application communicates with a centralized server that stores and manages shared data.

Three-Tier Architecture

Responsibilities are separated into:

- Presentation Layer
- Business Layer
- Data Layer

This separation improves maintainability and code organization.

Monolithic Architecture

All business modules are packaged and deployed together as a single application.

Microservices Architecture

Business capabilities are split into small, independent services that communicate through APIs.

1.3 Monolithic Architecture

What is a Monolithic Application?

A **Monolithic Application** is an application in which all business functionalities are developed, packaged, and deployed as a single executable unit.

Although the application may be divided into logical layers, all modules execute within the same process and share a common deployment lifecycle.

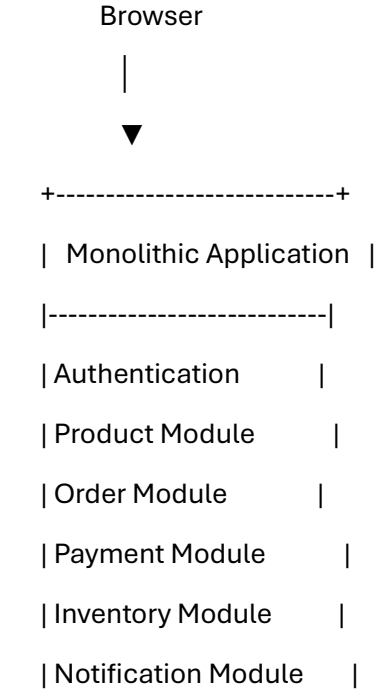
Example

An online shopping application may include:

- User Management
- Product Catalog
- Inventory
- Shopping Cart
- Orders
- Payments
- Shipping
- Notifications

In a monolithic application, all these modules are part of one project and are deployed together.

Monolithic Architecture



+----------------------------+

|

▼

Shared Database

All modules share:

- One codebase
- One deployment
- One database
- One runtime environment

Internal Layered Structure

A typical Spring Boot monolithic application follows a layered architecture.

Client

|

▼

Controller

|

▼

Service

|

▼

Repository

|

▼

Database

Controller Layer

- Receives HTTP requests.
- Validates input.
- Invokes business services.
- Returns responses.

Service Layer

- Contains business rules.
- Coordinates application logic.
- Manages transactions.

Repository Layer

- Interacts with the database.
- Performs CRUD operations.

Database Layer

Stores application data such as products, customers, and orders.

Request Flow

Consider the request to retrieve a product.

Client

|

GET /products/101

|

Controller

|

Service

|

Repository

|

Database

|

Repository

|

Service

|

Controller

|

JSON Response

All processing occurs within the same application process, making communication fast because no network calls occur between modules.

Advantages of Monolithic Architecture

- Simple to develop and understand.
- Easy to deploy as a single application.
- Fast internal method calls.
- Simplified debugging.
- Straightforward transaction management.
- Lower infrastructure requirements for small systems.

Limitations of Monolithic Architecture

As applications grow, several challenges emerge.

- Increasing codebase size
- Tight coupling between modules
- Longer build and deployment times
- Entire application must be redeployed for small changes
- Limited scalability
- Single point of failure
- Difficult adoption of new technologies

For small and medium-sized applications, these limitations may not be significant. However, for large enterprise systems serving millions of users, they become major obstacles.

1.4 Why Organizations Moved Beyond Monoliths

Imagine an online shopping platform during a festive sale.

The **Product Catalog** receives thousands of requests per second, while modules such as Notifications or Reports experience relatively low traffic.

In a monolithic application, scaling the Product Catalog requires deploying additional instances of the entire application, including all unrelated modules.

Similarly, if the Reporting module encounters a memory leak, it can negatively impact critical operations such as order processing or payment.

These limitations motivated organizations to adopt an architecture where each business capability could be developed, deployed, and scaled independently.

This architectural style is known as **Microservices**.

Module Summary

In this module, you learned:

- What enterprise applications are.
- How software architecture has evolved.
- The structure and characteristics of monolithic applications.
- The advantages and limitations of monolithic architecture.
- Why modern enterprise systems increasingly adopt microservices.

1.5 Microservices Architecture

What are Microservices?

A **Microservice** is a small, independent application that implements a **single business capability**. Unlike a monolithic application, where all functionalities are packaged together, each microservice is developed, tested, deployed, and scaled independently.

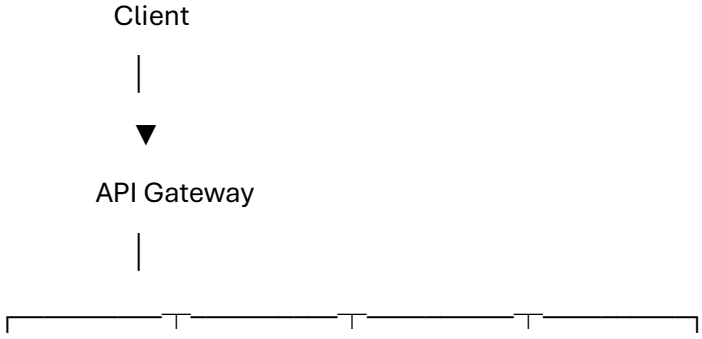
Each microservice has its own:

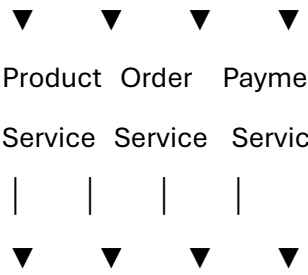
- Source code
- Business logic
- Database
- Deployment lifecycle
- Configuration

A complete application is formed by multiple microservices working together through APIs or messaging.

E-Commerce Example

Instead of building one large application, the system is divided into several services.





Each service is responsible for one business function.

Service	Responsibility
Product Service	Manage products
Order Service	Process orders
Inventory Service	Maintain stock
Payment Service	Process payments
Customer Service	Manage customers
Notification Service	Send emails and SMS

Characteristics of Microservices

1. Single Responsibility

Each service performs one business function.

Examples:

- Product Service
- Order Service
- Payment Service

This makes the code easier to understand and maintain.

---

2. Independent Deployment

Each service can be deployed without affecting other services.

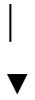
For example, updating the Product Service does not require redeploying the Payment Service.

---

3. Independent Database

Each service owns its database.

Product Service



ProductDB

Order Service

OrderDB

A service should **never** access another service's database directly.

Instead, it communicates using APIs or events.

---

4. Loose Coupling

Services interact through well-defined interfaces.

For example:

Order Service

HTTP Request

Payment Service

This allows services to evolve independently.

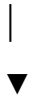
---

5. Independent Scaling

Suppose the Product Service receives much higher traffic than the Order Service.

Only the Product Service needs additional instances.

Load Balancer



Product Service

Product Service

Product Service

Order Service

Payment Service

This optimizes infrastructure usage.

---

6. Fault Isolation

If one service fails, the remaining services can continue operating.

For example:

- Notification Service fails
- Customers can still browse products.
- Orders can still be placed.
- Payments can still be processed.

This improves overall system availability.

1.6 Monolithic vs Microservices

Feature	Monolithic	Microservices
Deployment	Single application	Independent services
Codebase	Single	Multiple
Database	Shared	Database per service
Scaling	Entire application	Individual services
Technology Stack	Usually one	Can vary by service
Failure Impact	May affect entire application	Usually isolated
Team Structure	One large team	Multiple independent teams
Build Time	Longer	Faster
Deployment Frequency	Less frequent	Frequent
Cloud Readiness	Limited	Excellent

Choosing the Right Architecture

Choose Monolithic When

- Building a prototype or MVP.
- The development team is small.
- The application has limited business complexity.
- Rapid initial development is the priority.

Choose Microservices When

- The application is large and continuously evolving.
- Independent deployment is required.
- Different services need different scalability.
- Multiple teams work in parallel.
- The application is cloud-native.

There is no universally "best" architecture. The right choice depends on the application's size, complexity, team structure, and business requirements.

Module 1 Summary

In this module, you learned:

- The concept of Microservices.
- The key characteristics of Microservices.
- How Microservices differ from Monolithic Architecture.
- Situations where each architecture is most appropriate.

Module 2 – Spring Boot 4.0.1 Fundamentals

Learning Objectives

After completing this module, you will be able to:

- Understand the purpose of Spring Boot.
- Create a Spring Boot project.
- Understand the basic project structure.
- Explain the layered architecture of a Spring Boot application.

2.1 What is Spring Boot?

Spring Boot is an extension of the Spring Framework that simplifies the development of Java applications by providing sensible defaults and reducing configuration.

It enables developers to build **standalone, production-ready applications** quickly.

Instead of configuring an application from scratch, Spring Boot provides:

- Auto Configuration
- Starter Dependencies
- Embedded Web Server
- Externalized Configuration
- Production-ready features

Why Spring Boot?

Spring Boot is widely used for developing microservices because it:

- Reduces development time.
- Simplifies project setup.
- Supports REST API development.
- Integrates easily with databases.
- Works well with cloud platforms.

2.2 Spring Boot Architecture

A Spring Boot application typically follows a layered architecture.

Client

|

HTTP Request

|

Controller

|

Service

|

Repository

|

Database

Controller

Receives HTTP requests and returns HTTP responses.

Service

Contains business logic.

Coordinates application operations.

Repository

Interacts with the database using Spring Data JPA.

Database

Stores application data such as products, customers, and orders.

2.3 Creating a Spring Boot Project

We will create our project using **Spring Initializr**.

Project Settings

Property	Value
Project	Maven
Language	Java
Spring Boot	4.0.1
Java	21
Packaging	JAR

Dependencies

- Spring Web
- Spring Data JPA
- Validation
- MySQL Driver

2.4 Standard Project Structure

```
src
└── main
    ├── java
```

```
│   └── com.example.product
│
│       ├── controller
│       ├── service
│       ├── repository
│       ├── entity
│       ├── exception
│       └── ProductApplication
│
└── resources
    ├── application.properties
    └── static
```

Package Responsibilities

Package	Purpose
controller	REST Controllers
service	Business Logic
repository	Database Access
entity	JPA Entities
exception	Global Exception Handling

Organizing the project into packages improves readability and maintainability.

Module 2 Summary

In this module, you learned:

- What Spring Boot is and why it is used.
- The layered architecture of a Spring Boot application.
- How to create a Spring Boot project using Spring Initializr.
- The recommended package structure for enterprise applications.

Module 3 – REST API Fundamentals

Learning Objectives

After completing this module, you will be able to:

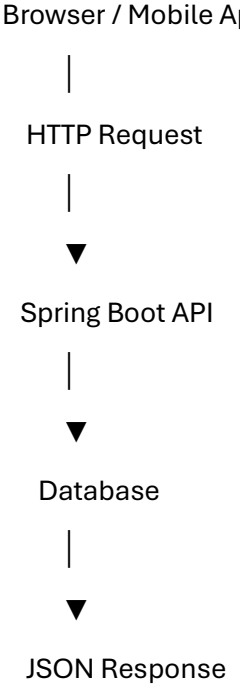
- Understand the REST architectural style.
- Use HTTP methods to perform CRUD operations.
- Design resource-oriented REST APIs.
- Exchange data using JSON.
- Use appropriate HTTP status codes.
- Design the REST APIs for the Product Service.

3.1 What is an API?

An **Application Programming Interface (API)** is a contract that enables two software applications to communicate with each other.

In a web application, the frontend (web browser or mobile app) communicates with the backend through APIs.

For example, when a user searches for a product in an e-commerce application, the frontend sends a request to the Product Service, which retrieves the required data from the database and returns it to the client.



3.2 What is REST?

REST (Representational State Transfer) is an architectural style for building web services that communicate using the HTTP protocol.

REST is based on the concept of **resources**.

Examples of resources include:

- Product
- Customer
- Order
- Payment
- Employee

Each resource is identified by a unique URL.

Examples:

/products

/orders

/customers

A client performs operations on these resources using standard HTTP methods.

3.3 HTTP Methods

REST APIs commonly use the following HTTP methods.

HTTP Method	Operation	Example
GET	Retrieve data	Get all products
POST	Create a resource	Add a product
PUT	Update a resource	Update a product
DELETE	Remove a resource	Delete a product

Examples

Retrieve all products

GET /products

Retrieve a specific product

GET /products/101

Create a new product

POST /products

Update an existing product

PUT /products/101

Delete a product

DELETE /products/101

3.4 Designing RESTful URLs

A well-designed REST API uses **nouns** to represent resources rather than verbs.

Good Examples

GET /products

GET /products/101

POST /products

PUT /products/101

DELETE /products/101

Poor Examples

/getProducts

/createProduct

/updateProduct

/deleteProduct

REST URL Guidelines

- Use nouns instead of verbs.
- Use plural names for collections (for example, /products).
- Use path variables to identify specific resources (for example, /products/{id}).
- Keep URLs short and meaningful.

3.5 JSON – Data Exchange Format

REST APIs typically exchange data using **JSON (JavaScript Object Notation)** because it is lightweight, human-readable, and supported by virtually all programming languages.

Product JSON

```
{
  "id": 101,
  "name": "Laptop",
  "category": "Electronics",
  "price": 65000,
  "quantity": 20
}
```

JSON Array

```
[
 {
   "id":101,
   "name":"Laptop"
 },
 {
   "id":102,
   "name":"Mouse"
 }
]
```

3.6 HTTP Status Codes

Every REST API returns an HTTP status code indicating the outcome of the request.

Status Code	Meaning	Typical Scenario
200 OK	Request successful	Data retrieved or updated
201 Created	Resource created	Product added successfully
204 No Content	Request successful with no response body	Product deleted
400 Bad Request	Invalid request	Validation failure

Status Code	Meaning	Typical Scenario
404 Not Found	Resource not found	Invalid Product ID
500 Internal Server Error	Unexpected server error	Application exception

Using appropriate status codes makes APIs easier to consume and debug.

3.7 Product Service API Design

The Product Service that we build in this course will expose the following REST endpoints.

Method	Endpoint	Description
GET	/products	Retrieve all products
GET	/products/{id}	Retrieve a product by ID
POST	/products	Create a new product
PUT	/products/{id}	Update an existing product
DELETE	/products/{id}	Delete a product

These endpoints follow REST principles and represent operations on the **Product** resource.

Module 3 Summary

In this module, you learned:

- The role of APIs in application communication.
- The fundamentals of REST architecture.
- The purpose of HTTP methods.
- How to design resource-oriented URLs.
- The role of JSON in REST APIs.
- The importance of HTTP status codes.
- The REST endpoints for the Product Service.

Lab 1 – Creating the Spring Boot Project

Objective

In this lab, you will create a Spring Boot 4.0.1 project that serves as the foundation for the Product Service.

By the end of this lab, you will have a runnable Spring Boot application ready for implementing REST APIs.

Prerequisites

Ensure the following software is installed:

Software	Version
Java	21
Maven	Latest
IntelliJ IDEA / Eclipse	Latest
MySQL	8.x
Postman	Latest

Step 1 – Create a New Spring Boot Project

Create the project using **Spring Initializr**.

Project Configuration

Property	Value
Project	Maven
Language	Java
Spring Boot	4.0.1
Group	com.example
Artifact	product-service
Packaging	JAR
Java Version	21

Step 2 – Add Required Dependencies

Select the following dependencies:

- Spring Web
- Spring Data JPA
- Validation

- MySQL Driver

These dependencies provide support for:

- REST API development
- Database access using JPA
- Input validation
- MySQL connectivity

Step 3 – Open the Project

After generating the project:

1. Extract the downloaded ZIP file.
2. Open it using IntelliJ IDEA or Eclipse.
3. Allow Maven to download the required dependencies.

Step 4 – Verify the Project Structure

The generated project should contain the following structure:

src

├── main

│ ├── java

│ └── resources

│

└── test

pom.xml

Inside the Java package, you will organize your application as follows:

com.example.product

├── controller

├── service

├── repository

├── entity

├── exception

└── ProductApplication

This layered structure separates concerns and makes the application easier to maintain.

Step 5 – Run the Application

Run the ProductApplication class from your IDE.

If the application starts successfully, you should see log messages indicating that the embedded web server has started.

Example:

Started ProductApplication in 2.8 seconds

Tomcat started on port 8080

The application is now ready for development.

Lab 2 – Creating Product Entity and Application Layers

Objective

In this lab, we will create the basic building blocks of the Product Service.

By the end of this lab, you will have:

- A database entity representing Product.
- Repository layer for database operations.
- Service layer containing business logic.
- Proper Spring Boot layered architecture.

4.1 Understanding Application Layers

A Spring Boot application commonly follows a layered architecture.

Client

|

|

REST Controller

|

|

Service Layer

|

|

Repository Layer

|

|

Database

Each layer has a specific responsibility.

Controller Layer

Responsible for:

- Receiving HTTP requests.
- Calling service methods.
- Returning HTTP responses.

Example:

GET /products

The controller receives the request and delegates processing to the service layer.

Service Layer

Responsible for:

- Business rules.
- Data processing.
- Application workflow.

Example:

Before saving a product:

- Check product details.
- Apply business validations.
- Call repository methods.

Repository Layer

Responsible for:

- Database communication.
- CRUD operations.

- Query execution.

Spring Data JPA provides repository interfaces that reduce database coding.

Entity Layer

Represents database tables as Java objects.

Example:

Database Table:

PRODUCT

ID

NAME

CATEGORY

PRICE

QUANTITY

Java Entity:

Product.java

4.2 Creating Package Structure

Inside:

com.example.product

Create the following packages:

com.example.product

├— controller

├— service

├— repository

├— entity

├── exception

└── ProductApplication

Why Use Packages?

A well-organized package structure:

- Improves maintainability.
- Separates responsibilities.
- Makes large applications easier to manage.
- Follows enterprise development practices.

4.3 Configure Database Connection

Open:

application.properties

Add MySQL configuration.

spring.datasource.url=jdbc:mysql://localhost:3306/productdb

spring.datasource.username=root

spring.datasource.password=sujata

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

Configuration Explanation

Database URL

spring.datasource.url

Defines the database location.

Example:

localhost:3306/productdb

Username and Password

spring.datasource.username

spring.datasource.password

Used to connect to MySQL.

Hibernate Configuration

spring.jpa.hibernate.ddl-auto=update

Automatically updates database tables based on entity changes.

Show SQL

spring.jpa.show-sql=true

Displays generated SQL queries in console.

4.4 Creating Product Entity

Create:

entity/Product.java

Product Entity Code

```
package com.example.product.entity;



import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;



@Entity
@Table(name="products")
```

```
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;

    private String category;

    private double price;

    private int quantity;

    public Product() {
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
```

```
}

public String getName() {
   return name;
}

public void setName(String name) {
   this.name = name;
}

public String getCategory() {
   return category;
}

public void setCategory(String category) {
   this.category = category;
}

public double getPrice() {
   return price;
}

public void setPrice(double price) {
   this.price = price;
}
```

```
    public int getQuantity() {
        return quantity;
    }


    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }
}
```

4.5 Understanding JPA Annotations

@Entity

Marks the class as a database entity.

Example:

@Entity

public class Product

Hibernate maps this class to a database table.

@Table

Defines the table name.

@Table(name="products")

Database table:

products

@Id

Defines the primary key.

@Id

private Long id;

@GeneratedValue

Automatically generates ID values.

@GeneratedValue(strategy = GenerationType.IDENTITY)

Example:

1

2

3

4

4.6 Creating Repository Layer

Create:

repository/ProductRepository.java

Repository Code

```
package com.example.product.repository;



import org.springframework.data.jpa.repository.JpaRepository;

import com.example.product.entity.Product;



public interface ProductRepository
    extends JpaRepository<Product, Long> {

}
```

Understanding JpaRepository

Spring Data JPA provides built-in CRUD operations.

Because ProductRepository extends JpaRepository, we automatically get:

Method	Purpose
save()	Insert/Update
findAll()	Retrieve all records
findById()	Retrieve by ID
deleteById()	Delete record
count()	Count records

No SQL code is required.

4.7 Creating Service Layer

The service layer contains business logic between the controller and repository.

Create:

service/ProductService.java

Service Interface

```
package com.example.product.service;



import java.util.List;

import com.example.product.entity.Product;



public interface ProductService {

    Product saveProduct(Product product);

    List<Product> getAllProducts();
```

```
    Product getProductById(Long id);


    void deleteProduct(Long id);

}
```

4.8 Service Implementation

Create:

service/ProductServiceImpl.java

```
package com.example.product.service;


import java.util.List;

import org.springframework.stereotype.Service;

import com.example.product.entity.Product;
import com.example.product.repository.ProductRepository;


@Service
public class ProductServiceImpl
        implements ProductService {


    private ProductRepository repository;
```

```
public ProductServiceImpl(ProductRepository repository){
   this.repository = repository;
}



@Override
public Product saveProduct(Product product){

   return repository.save(product);

}



@Override
public List<Product> getAllProducts(){

   return repository.findAll();

}



@Override
public Product getProductById(Long id){

   return repository.findById(id)
         .orElse(null);

}
```

```
    @Override
    public void deleteProduct(Long id){

        repository.deleteById(id);

    }

}
```

4.9 Understanding Spring Annotations

@Service

Indicates that the class contains business logic.

@Service

public class ProductServiceImpl

Spring automatically creates an object of this class.

Dependency Injection

Instead of creating objects manually:

new ProductRepository()

Spring provides required objects automatically.

Example:

public ProductServiceImpl(

ProductRepository repository)

{

this.repository = repository;

}

This is called **Constructor Injection**.

Application Flow After This Step

Client

|

|

Product Controller

|

|

Product Service

|

|

Product Repository

|

|

MySQL Database

Checkpoint

At this stage, the application contains:

- ✅ Spring Boot Project
- ✅ Database Configuration
- ✅ Product Entity

✅ Repository Layer
✅ Service Layer

The next step is to expose these operations through REST APIs.

Lab 3 – Building REST Controller and CRUD APIs

Objective

In this lab, we will expose the Product Service functionality through REST APIs.

By the end of this lab, you will have:

- REST Controller
- CRUD REST endpoints
- Request and Response handling
- API testing using Postman
- Validation and exception handling basics

5.1 Creating Product Controller

Create the following package:

controller

Create:

ProductController.java

Product Controller Code

```
package com.example.product.controller;



import java.util.List;


import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;


import com.example.product.entity.Product;
```

```
import com.example.product.service.ProductService;



@RestController
@RequestMapping("/products")
public class ProductController {


  private ProductService service;



  public ProductController(ProductService service){
    this.service = service;
  }




  @PostMapping
  public ResponseEntity<Product> saveProduct(
      @RequestBody Product product){

    Product savedProduct =
        service.saveProduct(product);

    return new ResponseEntity<>(
        savedProduct,
        HttpStatus.CREATED);

  }
```

```
@GetMapping
public ResponseEntity<List<Product>> getAllProducts(){

   return ResponseEntity.ok(
         service.getAllProducts());

}




@GetMapping("/{id}")
public ResponseEntity<Product> getProductById(
      @PathVariable Long id){

   Product product =
         service.getProductById(id);



   if(product == null){
      return ResponseEntity.notFound().build();
   }



   return ResponseEntity.ok(product);

}




@PutMapping("/{id}")
```

```
    public ResponseEntity<Product> updateProduct(
            @PathVariable Long id,
            @RequestBody Product product){

        product.setId(id);

        Product updated =
                service.saveProduct(product);



        return ResponseEntity.ok(updated);

    }




    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteProduct(
            @PathVariable Long id){

        service.deleteProduct(id);

        return ResponseEntity.noContent().build();

    }

}
```

5.2 Understanding Controller Annotations

@RestController

Indicates that this class handles REST requests.

@RestController

public class ProductController

It combines:

@Controller

@ResponseBody

Responses are automatically converted into JSON.

@RequestMapping

Defines the base URL.

@RequestMapping("/products")

All APIs start with:

/products

@GetMapping

Handles GET requests.

Example:

@GetMapping

URL:

GET /products

@PostMapping

Handles POST requests.

Example:

@PostMapping

URL:

POST /products

@PutMapping

Handles update operations.

PUT /products/101

@DeleteMapping

Handles delete operations.

DELETE /products/101

@RequestBody

Converts incoming JSON into Java objects.

Example Request:

{

"name":"Laptop",

"category":"Electronics",

"price":65000,

"quantity":10

}

Converted into:

Product product

@PathVariable

Reads values from URL.

Example:

GET /products/101

Code:

@PathVariable Long id

Value:

id = 101

5.3 Testing CRUD APIs

We will test APIs using Postman.

API 1 – Create Product

Request

POST http://localhost:8080/products

Body:


```
{
  "name":"Laptop",
  "category":"Electronics",
  "price":65000,
  "quantity":20
}
```

Response

Status:

201 CREATED

Response:


```
{
  "id":1,
  "name":"Laptop",
  "category":"Electronics",
  "price":65000,
  "quantity":20
}
```

API 2 – Get All Products

Request

GET http://localhost:8080/products

Response

Status:

200 OK

Response:


```
[
 {
  "id":1,
```

```
  "name":"Laptop",
  "category":"Electronics",
  "price":65000,
  "quantity":20
 }
]
```

API 3 – Get Product By ID

Request

GET /products/1

Response


```
{
"id":1,
"name":"Laptop",
"category":"Electronics",
"price":65000,
"quantity":20
}
```

API 4 – Update Product

Request

PUT /products/1

Body:


```
{
"name":"Gaming Laptop",
"category":"Electronics",
"price":85000,
"quantity":15
}
```

Response

200 OK

API 5 – Delete Product

Request

DELETE /products/1

Response

204 NO CONTENT

Lab 4 – Adding Validation

Why Validation?

Validation ensures that incorrect data does not enter the application.

Examples:

- Product name cannot be empty.
- Price cannot be negative.
- Quantity should be positive.

Add Validation Dependency

Spring Boot includes Jakarta Validation support.

Entity:

import jakarta.validation.constraints.NotBlank;

import jakarta.validation.constraints.Positive;

Update Product Entity

@NotBlank(message="Product name is required")

private String name;

@Positive(message="Price must be positive")

private double price;

@Positive(message="Quantity must be positive")

private int quantity;

Update Controller

Add:

import jakarta.validation.Valid;

Change:

public ResponseEntity<Product> saveProduct(

@Valid @RequestBody Product product)

Now invalid requests will automatically be rejected.

Lab 5 – Global Exception Handling

Why Exception Handling?

Without proper handling, users receive technical error messages.

Example:

NullPointerException

SQL Exception

Stack Trace

Instead, APIs should return meaningful responses.

Create Exception Package

exception

Create:

GlobalExceptionHandler.java

Code

package com.example.product.exception;

```
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;


@RestControllerAdvice
public class GlobalExceptionHandler {


   @ExceptionHandler(Exception.class)
   public ResponseEntity<String>
   handleException(Exception ex){

      return new ResponseEntity<>(
            ex.getMessage(),
            HttpStatus.INTERNAL_SERVER_ERROR);

   }

}
```

Understanding @RestControllerAdvice

It provides centralized exception handling.

Benefits:

- Cleaner controllers
- Consistent error responses
- Easier maintenance

Day 1 Complete Application Flow

Client

|

REST Controller

|

Service Layer

|

Spring Data JPA

|

MySQL DB

Final Project Structure

product-service

src/main/java

com.example.product

```
│
├── controller
│   └── ProductController
│
├── service
│   ├── ProductService
│   └── ProductServiceImpl
│
```

├── repository

│ └── ProductRepository

│

├── entity

│ └── Product

│

├── exception

│ └── GlobalExceptionHandler

│

└── ProductApplication

Day 1 Exercises

Exercise 1

Add a new field:

brand

to the Product entity.

Exercise 2

Create a new API:

GET /products/category/{category}

to search products by category.

Exercise 3

Add validation:

- Product name minimum length: 3 characters.
- Price should be greater than 100.

Exercise 4

Create a custom error response:

{

"timestamp":"",

"message":"",

"status":400

}

Day 1 Summary

Today you learned the complete foundation required for building microservices using Spring Boot.

You started with:

Architecture Concepts

Monolithic

↓

Microservices

Then moved to:

Spring Boot Development

Controller

↓

Service

↓

Repository

↓

Database

Finally, you built:

Product REST Microservice

REST API

+

Spring Boot

+

Spring Data JPA

+

MySQL
